
Next, we create the `COLUMN_LABELS` constant:

```
COLUMN_LABELS = ("1", "2", "3", "4", "5", "6", "7")
assert len(COLUMN_LABELS) == BOARD_WIDTH
```

We'll use this constant later to ensure the player selects a valid column. Note that if we ever set `BOARD_WIDTH` to a value other than 7, we'll have to add labels to or remove labels from the `COLUMN_LABELS` tuple. I could have avoided this by generating the value of `COLUMN_LABELS` based on `BOARD_WIDTH` with code like this: `COLUMN_LABELS = tuple([str(n) for n in range(1, BOARD_WIDTH + 1)])`. But `COLUMN_LABELS` is unlikely to change in the future, because the standard Four-in-a-Row game is played on a 7 by 6 board, so I decided to write out an explicit tuple value.

Sure, this hardcoding is a code smell, as described in “Magic Numbers” on page 71, but it's more readable than the alternative. Also, the `assert` statement warns us about changing `BOARD_WIDTH` without updating `COLUMN_LABELS`.

As with Tower of Hanoi, the Four-in-a-Row program uses ASCII art to *draw* the game board. The following lines are a single assignment statement with a multiline string:

```
# The template string for displaying the board:
BOARD_TEMPLATE = """
    1234567
+-----+
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
|{}{}{}{}{}{}|
+-----+"""
```

This string contains braces (`{}`) that the `format()` string method will replace with the board's contents. (The `displayBoard()` function, explained later, will handle this.) Because the board consists of seven columns and six rows, we use seven brace pairs `{}` in each of the six rows to represent every slot. Note that just like `COLUMN_LABELS`, we're technically hardcoding the board to create a